Experiment No: 06

Aim: To Deploy and Manage Containers Using Kubernetes.

DOP: **DOS:**

Theory:

Kubernetes

Kubernetes is an open-source **container orchestration platform** used to deploy, manage, scale, and maintain containerized applications. It automates tasks such as creating containers, maintaining the required number of application instances, providing networking, and replacing failed application instances.

In this experiment, Kubernetes is enabled through **Docker Desktop** and is used to deploy an NGINX web server.

Container

A container is a lightweight, isolated environment that contains an application and its required dependencies. Containers allow applications to run consistently across different environments.

In this experiment, the **NGINX container image** is used.

Pod

A **Pod** is the smallest deployable unit in Kubernetes. A Pod contains one or more containers. In this experiment, each Pod contains an NGINX container.

Kubernetes
↓
Pod
↓
NGINX Container

Deployment

A **Deployment** manages and maintains a desired number of Pods. It allows Kubernetes to create, replace, and scale application Pods.

In this experiment: **kubectl create deployment myapp --image=nginx**
creates a Deployment named myapp using the NGINX image.

Initially:
Deployment
↓
1 Pod
↓
NGINX Container

Service

A **Service** provides a stable network endpoint through which an application can be accessed. Since Pod IP addresses can change, a Service provides a consistent way to communicate with the Pods.

In this experiment: **kubectl expose deployment myapp --type=NodePort --port=80**
creates a Service for the Deployment.

Browser
↓

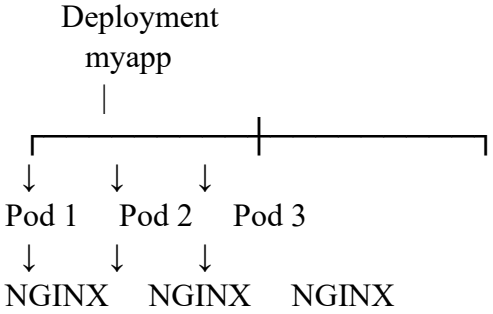
Service
↓
NGINX Pod
↓
NGINX Container

kubectl
kubectl is the command-line tool used to interact with a Kubernetes cluster.
It can be used to:

- Create resources
- View Pods and Deployments
- Create Services
- Scale applications
- Inspect resources
- Delete resources

Examples:
kubectl get pods
kubectl get deployments
kubectl get services
kubectl describe deployment myapp

Scaling
Scaling means changing the number of application instances.
The Deployment is scaled to three replicas using: **kubectl scale deployment myapp --replicas=3**
After scaling:



Kubernetes attempts to maintain the requested three replicas.

Port Forwarding
Port forwarding creates a temporary connection between a local computer port and a Kubernetes Service or Pod.
kubectl port-forward service/myapp 8090:80
Here:

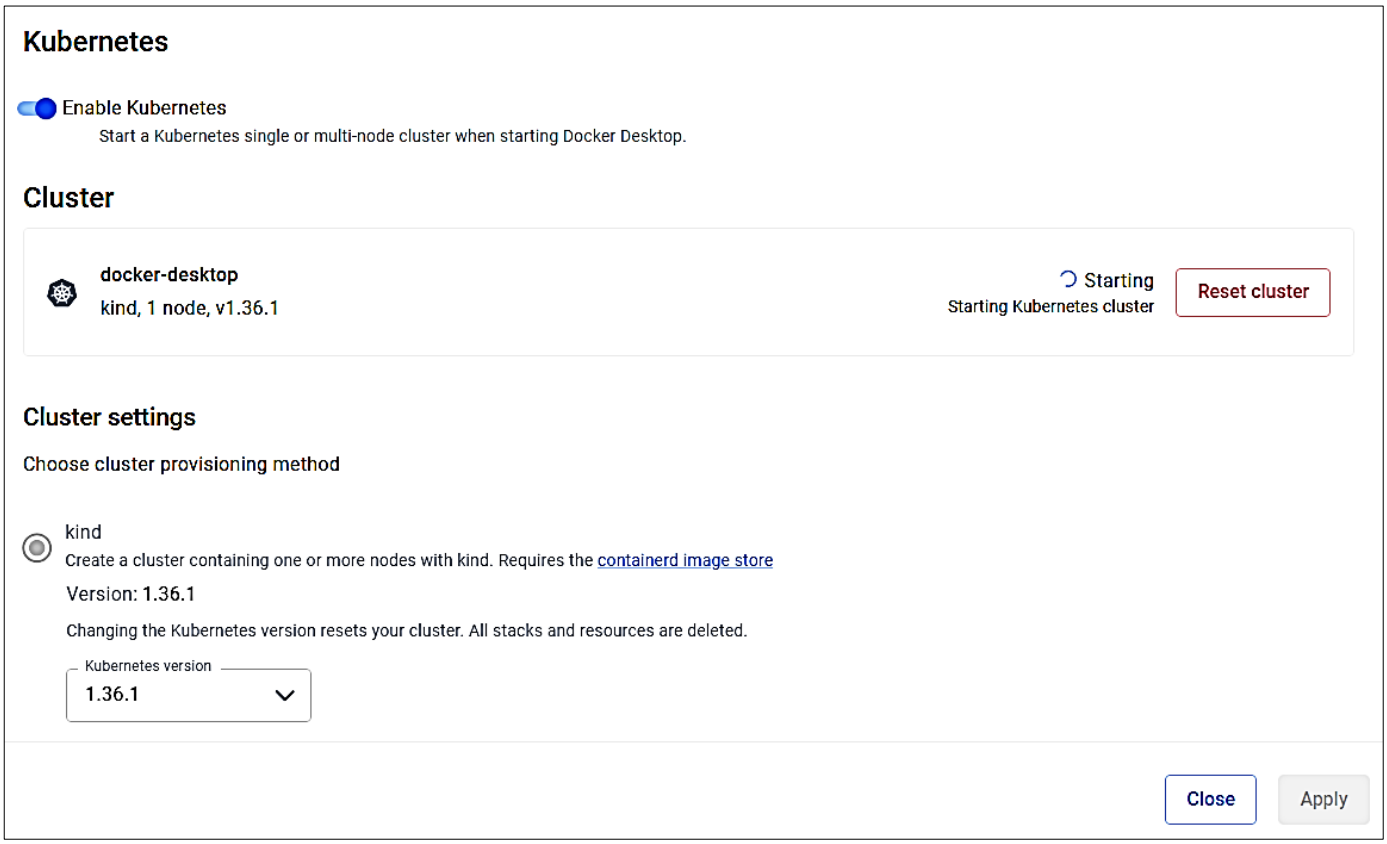
- 8090 is the local port.
- 80 is the Service port.

The application can then be accessed through: **http://localhost:8090**

3. Procedure
Step 1 – Install and Enable Kubernetes

1. Install Docker Desktop.
2. Start Docker Desktop.
3. Go to **Settings → Kubernetes**.

4. Enable Kubernetes.
5. Click **Apply & Restart**.
6. Wait until Kubernetes shows as running/ready.



Step 2 – Verify Kubernetes Using PowerShell
Open PowerShell and execute: **kubectl version**
This verifies that **kubectl** is installed and can communicate with Kubernetes.

```
PS C:\Users\Aaditya> kubectl version
Client Version: v1.36.1
Kustomize Version: v5.8.1
Server Version: v1.36.1
PS C:\Users\Aaditya>
```

Step 3 – Create Kubernetes Deployment
Create an NGINX Deployment: **kubectl create deployment myapp --image=nginx**
Check the Deployment: **kubectl get deployments**
Check the Pod: **kubectl get pods**

```
PS C:\Users\Aaditya> kubectl create deployment myapp --image=nginx
deployment.apps/myapp created
PS C:\Users\Aaditya> kubectl get deployments
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
myapp   1/1     1            1           22s
PS C:\Users\Aaditya> kubectl get pods
NAME                     READY   STATUS    RESTARTS   AGE
myapp-68db9dd855-q5z27   1/1     Running   0          32s
PS C:\Users\Aaditya>
```

Step 4 – Expose the Application

Create a Kubernetes Service: **kubectl expose deployment myapp --type=NodePort --port=80**

Check the Service: **kubectl get services**

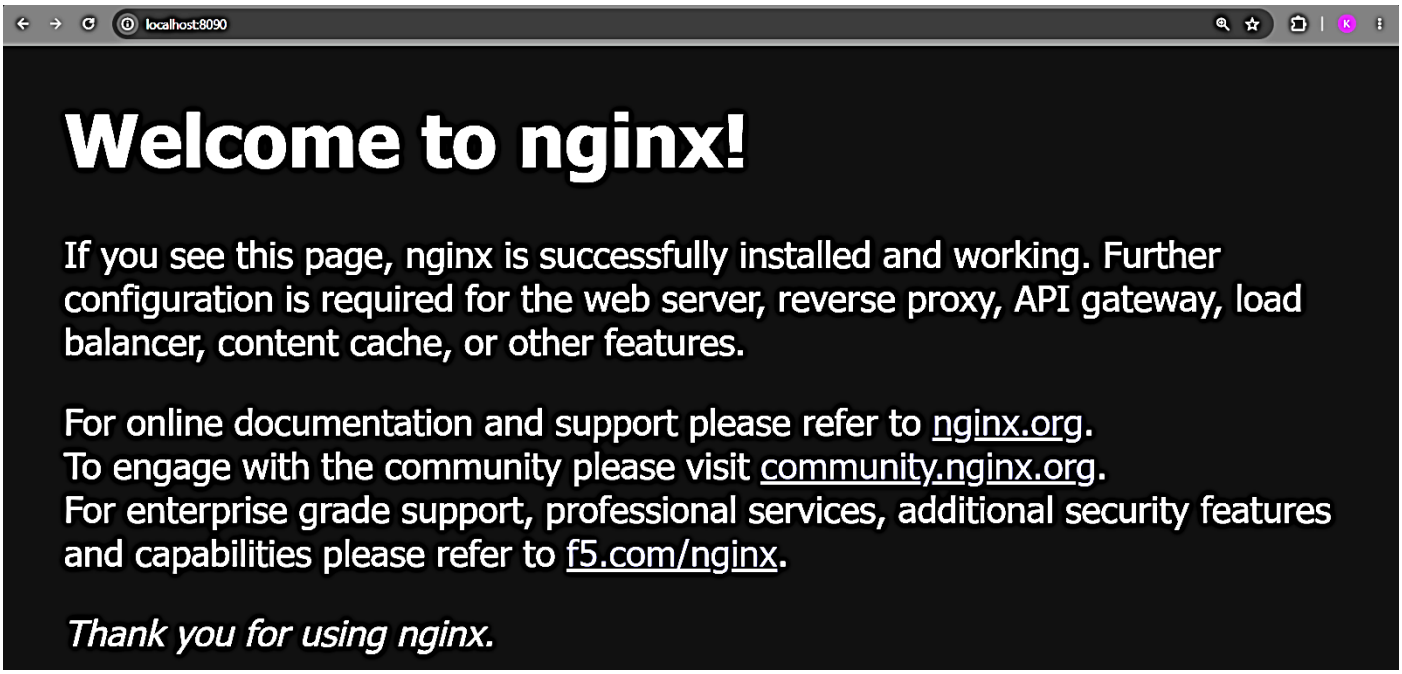
```
PS C:\Users\Aaditya> kubectl expose deployment myapp --type=NodePort --port=80
service/myapp exposed
PS C:\Users\Aaditya> kubectl get services
NAME         TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)       AGE
kuberentes   ClusterIP   10.96.0.1      <none>        443/TCP       6m43s
myapp        NodePort    10.96.133.18   <none>        80:31521/TCP  20s
PS C:\Users\Aaditya>
```

Step 5 – Access the Application

Run: **kubectl port-forward service/myapp 8090:80**

You should get output similar to: **Forwarding from 127.0.0.1:8090 -> 80**

Now open the browser and enter: **http://localhost:8090**



Step 6 – Scale the Application

Initially check the Pods: **kubectl get pods**

Scale the Deployment to three replicas: **kubectl scale deployment myapp --replicas=3**

Check the Pods again: **kubectl get pods**

```
PS C:\Users\Aaditya> kubectl scale deployment myapp --replicas=3
deployment.apps/myapp scaled
PS C:\Users\Aaditya> kubectl get pods
NAME                     READY   STATUS    RESTARTS   AGE
myapp-68db9dd855-q5z27   1/1     Running   0          7m26s
myapp-68db9dd855-qz9fh   1/1     Running   0          9s
myapp-68db9dd855-xw5pc   1/1     Running   0          9s
PS C:\Users\Aaditya>
```

Step 7 – Check Deployment Details

Display detailed information about the Deployment: **kubectl describe deployment myapp**

You can also inspect the Pods: **kubectl describe pods**

The describe command provides detailed information about the resource, including its configuration, replicas, containers, events, conditions, and other Kubernetes information.

```
PS C:\Users\Aaditya> kubectl describe deployment myapp
Name:                   myapp
Namespace:              default
CreationTimestamp:      Wed, 23 Sep 2026 21:28:49 +0530
Labels:                 app=myapp
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               app=myapp
Replicas:               3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyTyps:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=myapp
  Containers:
    nginx:
      Image:        nginx
      Port:         <none>
      Host Port:    <none>
      Environment:  <none>
```

```
PS C:\Users\Aaditya> kubectl describe pods
Name:                 myapp-68db9dd855-q5z27
Namespace:            default
Priority:             0
Service Account:      default
Node:                 desktop-control-plane/172.18.0.2
Start Time:           Wed, 23 Sep 2026 21:28:49 +0530
Labels:               app=myapp
                      pod-template-hash=68db9dd855
Annotations:          <none>
Status:               Running
IP:                   10.244.0.6
IPs:
  IP:                 10.244.0.6
Controlled By:        ReplicaSet/myapp-68db9dd855
Containers:
  nginx:
    Container ID:     containerd://5821bf3c21bd72128346456ef9f1e03f97999d885f1c1648091b8ecbee092d39126
    Image:            nginx
    Image ID:         docker.io/library/nginx@sha256:abe47724e66aeab9a345d8e46a221c2fa8953c7848bb4a3bdbd9976
    Port:             <none>
    Host Port:        <none>
    State:            Running
      Started:        Wed, 23 Sep 2026 21:28:53 +0530
    Ready:            True
    Restart Count:    0
    Environment:      <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-2frj7 (ro)
```

Step 8 – Delete the Deployment and Service

After completing the experiment, delete the Deployment: **kubectl delete deployment myapp**

Delete the Service: **kubectl delete service myapp**

Verify:

kubectl get deployments

kubectl get services

The myapp Deployment and Service should no longer be present.

```
PS C:\Users\Aaditya> kubectl delete service myapp
service "myapp" deleted from default namespace
PS C:\Users\Aaditya> kubectl get deployments
No resources found in default namespace.
PS C:\Users\Aaditya> kubectl get services
NAME         TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
kubernetes   ClusterIP   10.96.0.1      <none>        443/TCP        17m
myapp        NodePort    10.96.133.18   <none>        80:31521/TCP   20s
PS C:\Users\Aaditya>
```

Conclusion: